

TP₁-valid Transformation Functions for Operations on ordered n -ary Trees

Tim Jungnickel and Tobias Herb
TU Berlin

Collaborative work on shared documents was revolutionized by web services like Google Docs or Etherpad. Multiple users can work on the same document in a comfortable and distributed way. For the synchronization of the changes a replication system named Operational Transformation is used. Such a system consists of a control algorithm and a transformation function. In essence, a transformation function solves the conflicts that arise when multiple users change the document at the same time. In this work we investigate on the correctness of such transformation functions. We introduce transformation functions n -ary trees that we designed especially for the purpose of synchronization changes on JSON objects. We provide a detailed proof of the necessary property: the Transformation Property 1.

1 Introduction

The collaborative work on shared documents has become extraordinary comfortable by web services like Google Docs¹ or Etherpad². Multiple users can edit a shared document at the same time and all changes will be automatically synchronized in the background. As comfortable the usage of such services is, as challenging and non trivial is the synchronization process in the background. Users demand a consistent result i.e., no matter which user modifies the document, the result after the synchronization must identical for all users. An optimistic replication system named *Operational Transformation* (OT) is used to guarantee that all replicas are consistent. Within this work extend our approach we present in [5] and provide the missing preliminaries that are necessary to build a fully working OT synchronization of JSON objects.

An OT system consists of two core components: a control algorithm and a transformation function [11]. A control algorithm handles the communication

¹<https://docs.google.com>

²<http://etherpad.org/>

between the clients and the exchange of the changes. A transformation function solves the conflicts that occur when two clients independently edit a document at the same time.

To give some intuition of the OT mechanism, we demonstrate OT with a simple text editing scenario. The sites S_1 , S_2 replicate the character sequence XYZ. S_1 inserts character A at position 0, resulting in AXYZ. S_2 simultaneously deletes the character Y at position 1, resulting in XZ. The operations are exchanged between the sites. If the remote operations are applied naively, then we get inconsistent replicas: S_1 results in AYZ and S_2 results in AXZ. Therefore, a transformation of the remote operation is performed, before the operation is applied. At S_1 , the position of the remote delete operation is incremented with respect to the local insert operation. At S_2 , the remote insert operation does not need to be transformed since the local delete operation has no effect on the remote insert operation. The transformations of the position of simultaneous operations leads to consistent replicas.

The major problems of applying linear OT on hierarchical structures are simultaneous inserts of hierarchical nodes. This can be well exemplified by an HTML example. We reuse our document containing the sequence XYZ, replicated on the sites S_1 , S_2 . Site S_1 decides that character Y should be in bold. This can be represented with two insert operations, producing the document state XYZ. Simultaneously S_2 decides that character Y should be in italic. If the server receives S_2 's edit after S_1 , it would need to transform the operations of S_2 against the operations of S_1 . Usually the server would be configured to place the later edit behind the first edit. If the operations of S_1 are applied before the transformed operations of S_2 are applied, then the document will be syntactically incorrect: $X<i>Y</i>Z$.

Contributions The contributions of this work are the following:

- We verify the necessary properties of a common transformation function for operations on lists with a Isabelle/HOL proof.
- We introduce a transformation function for operations on n -ary trees that is designed to support simultaneous editing of JSON objects in a programming language near notation.
- We verify the necessary properties of the introduced transformation function with a detailed proof.

2 Related Work

Davis et al [1] were, to our knowledge, the first that applied the OT approach on treelike structures. They extended operational transformation to support synchronous collaborative editing of documents written in dialects of SGML (Standard General Markup Language) such as XML and HTML. They introduced a set of structural operations with their associated transformation functions tailored for

SGML’s abstract data model grove. Their approach is followed by [9, 10] showing improvements in XML editing and implementations in collaborative business software. In our work we provide an alternative, more generic transformation function that is not shaped for XML. Hence with our transformation function we enable more general use cases of hierarchical OT. From this we can easily derive a transformation of operations on JSON objects. In addition we present our transformation functions in a programming language near notation so that they can be implemented easier.

Oster et al [7] proposed a framework for supporting collaborative writing of XML documents. Their framework works similarly to the Copy-Modify-Merge paradigm widely used in version control systems such as CVS. The synchronization of the replicated XML documents is based on Operational Transformation. They make also use of positional addressing scheme of the XML elements, comparable to our approach. They claim with respect to proving the correctness: “It is nearly impossible to do this by hand” and refer to an automated tool named VOTE to fulfill the challenge [4]. Compared to their work we define the underlying model precisely for our purpose — the synchronization of changes on generic hierarchical objects. Moreover we took the challenge of Oster et al and managed to prove the correctness *by hand*.

Our approach is most comparable with the approach of Imine et al [3]. They analyzed the correctness of common transformation functions for lists with respect to the necessary transformation properties with SPIKE. Based on their observations they propose a new transformation function with the desired properties that is easier to read and to prove. We also took one common transformation function for lists and used the interactive theorem prover Isabelle/HOL to verify one transformation property. Based on the gained experience we proposed a new transformation function for operations on n -ary trees that is, in contrast to the described work in the previous paragraphs, easier to understand, prove and implement.

3 Technical Preliminaries

In this section we present the necessary preliminaries to define a transformation function for tree operations.

3.1 Lists

We consider lists as one of the simplest form of a linear data structure. A list is recursively defined either as an *empty list* or as a *cell* containing an item and another list. In Table 1 we introduce a custom notation for lists similar to the notation in the programming language Python. Based on this notation we define an insert and a delete operation for lists in Def. 1 and Def. 2.

Definition 1 (insert_L). *The operation insert_L has three parameters: an item i , a position k and a list L with $k \leq |L|$. As result, the item i is inserted into the list L at position k :*

$$\text{insert}_L(i, k, L) \triangleq L[< k] + [i] + L[\geq k]$$

Table 1: Notation for lists based on the programming language Python.

Notation	Description
$[]$	Empty List and Delimiters We use $[$ and $]$ as delimiters for a list. Hence the list $[x, y, z]$ contains the items x , y and z . We denote the empty list as $[]$.
$ L $	Length of a List We define the length of a list L as the number of items in the list, denoted as $ L $.
$L[n]$	List Access Let L be an arbitrary list. We denote the access to the n^{th} element as $L[n]$. Note that $L[n]$ is only defined if $n < L $. We access the first element with $L[0]$.
$L[x, y]$	Intervals Let L be an arbitrary list. We write $L[x, y]$ for a new list that contains all elements from $L[x]$ to $L[y]$. Note that $L[x, y]$ can be the empty list if $x > y$ or the if x and y referencing to non existing elements. If only y is referencing to a non existing element, $L[x, y]$ contains all elements from $L[x]$ to the last element of L .
$L_1 + L_2$	List Concatenation Let L_1 and L_2 be two arbitrary lists. We define the concatenation of L_1 and L_2 , denoted as $L_1 + L_2$, as a new list that starts with L_1 and ends with L_2 .
$L_1 \subseteq L_2$	Sublists and Strict Sublists Let L_1 and L_2 be two arbitrary lists. We call L_1 a <i>sublist</i> of L_2 , denoted as $L_1 \subseteq L_2$, if L_2 starts with L_1 . If $L_1 \subseteq L_2$ and $ L_1 < L_2 $, we call L_1 a <i>strict sublist</i> of L_2 , denoted as $L_1 \subset L_2$.
$L_1 \subset L_2$	
$L[\leq x]$ $L[< x]$	Head and Tail Let L be an arbitrary list. We write $L[\leq x]$ for a new (sub)list that contains all elements from the first element to $L[x]$. We use the abbreviation $L[< x]$ for $L[\leq x - 1]$. The lists $L[\geq x]$ and $L[> x]$ are defined analogously.

Definition 2 (delete_L). *The operation delete_L has two parameters: a position k and a list L with $k < |L|$. As result, the item at position k is deleted from L :*

$$\text{delete}_L(k, L) \triangleq L[< k] + L[> k]$$

3.2 Trees

We consider n -ary trees with the simplest set of operations insert_T and delete_T . An n -ary tree is recursively defined as a pair of a value and a list of trees. A leaf is defined as a pair of a value and an empty list. Thus a tree cannot be empty, the smallest tree is a single leaf. As shown in Fig. 1, we use a list of natural numbers (called *access path*) to access the tree at a specific position. For a tree T and an access path pos we write $T[\![pos]\!]$ to get the *subtree* at position pos . We define the operations insert_T and delete_T in Def. 3 and Def. 4.

Definition 3 (insert_T). *The operation insert_T has three input parameters: a tree t , a non empty access path pos and a tree $T = (v, L)$. As result, the tree t will be inserted into T at position pos . We define the operation recursively:*

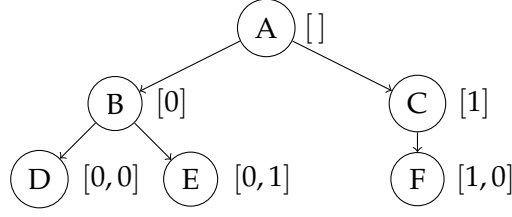


Figure 1: Tree representation and node access

$$\text{insert}_T(t, [x], (v, L)) \triangleq (v, \text{insert}_L(t, x, L))$$

$$\text{insert}_T(t, [x] + xs, (v, L)) \triangleq (v, \text{insert}_L(\text{insert}_T(t, xs, L[x]), x, \text{delete}_L(x, L)))$$

Definition 4 (delete_T). *The operation delete_T has two input parameters: a non empty access path pos and a tree T = (v, L). As result, the subtree at position pos will be deleted from T. We define the operation recursively:*

$$\text{delete}_T(t, [x], (v, L)) \triangleq (v, \text{delete}_L(x, L))$$

$$\text{delete}_T(t, [x] + xs, (v, L)) \triangleq (v, \text{insert}_L(\text{delete}_T(xs, L[x]), x, \text{delete}_L(x, L)))$$

According to our definition of trees, the second last element of an access path determines the node where a subtree should be inserted into or where a subtree should be deleted from. The last element of an access path determines the position inside the list of subtrees of the node at the second last element. We simply use the operation insert_L to insert a tree into the list of subtrees and we use delete_L to delete a tree from the list of subtrees.

We notice that both definitions of insert_T and delete_T are insufficient if the input access path directs to a non existing node. For the rest of the paper we assume, that we have a *valid* input for both operations i.e., the access path directs to a position where we can apply an insertion or a deletion. For a later setup in a server/client architecture we can easily check on both sides if an operation is valid. We define the validity of tree operations more precisely in Def. 5.

Definition 5 (Valid Tree Operations). *Let O be a tree operation on the tree T. We call O a valid tree operation if:*

- **case 1:** $O = \text{insert}_T(t, pos, T)$, then:

Up to the second last element the position parameter pos directs to an existing subtree (v, L) in T and the last element of pos is a valid position parameter for an insert_L operation in L.

- **case 2:** $O = \text{delete}_T(pos, T)$, then:

The position parameter pos directs to an existing subtree in T.

An insert_T operation can be considered for any strict sublist of the access path. As demonstrated in Fig. 2, the original insert_T operation on the left of the equation can also be seen as an insert_T operation with a sublist of the original access path, as long as the emerging subtree (visualized as thick circles and arrows) and

the deleted subtree (visualized as dotted circles and arrows) are adjusted. We formalize this observation in Lemma 1. Lemma 2 is the counterpart for delete_T to Lemma 1.

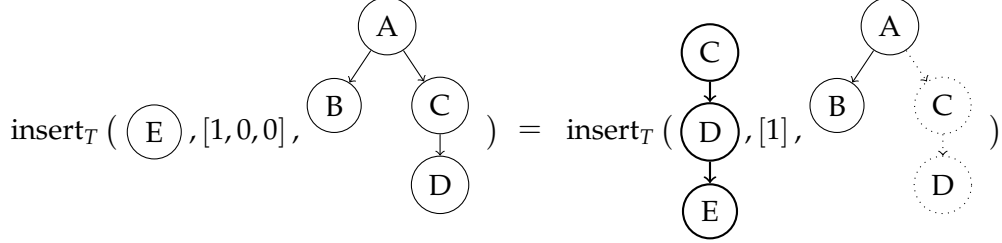


Figure 2: Example for Lemma 1

Lemma 1. *Given a tree T and a valid access path pos for an insert_T operation. For all $0 < i < |pos|$ and for all t the following statement is always true:*

$$\begin{aligned} \text{insert}_T(t, pos, T) = \\ \text{insert}_T(\text{insert}_T(t, pos[\geq i], T[pos[< i]]), pos[< i], \text{delete}_T(pos[< i], T)) \end{aligned}$$

Proof of Lemma 1. We prove Lemma 1 by induction over i .

base case: We fix an access path pos and a tree t and show the base case for $i = 1$ with $i < |pos|$ directly with the presented definitions. We assume pos is a valid access path for an insert_T operation on $T = (v, L)$.

$$\begin{aligned} & \text{insert}_T(t, pos, T) \\ \stackrel{D3}{=} & (v, \text{insert}_L(\text{insert}_T(t, pos[\geq 1], L[pos[0]]), pos[0], \text{delete}_L(pos[0], L)) \\ \stackrel{DT}{=} & (v, \text{insert}_L(\text{insert}_T(t, pos[\geq 1], T[pos[< 1]]), pos[0], \text{delete}_L(pos[0], L)) \\ \stackrel{D3}{=} & \text{insert}_T(\text{insert}_T(t, pos[\geq 1], T[pos[< 1]]), pos[< 1], (v, \text{delete}_L(pos[0], L))) \\ \stackrel{D4}{=} & \text{insert}_T(\text{insert}_T(t, pos[\geq 1], T[pos[< 1]]), pos[< 1], \text{delete}_T(pos[< 1], T)) \end{aligned}$$

inductive step: We fix an access path pos and a tree t and show the inductive step with $(i + 1) < |pos|$ directly with the presented definitions and the induction hypothesis. We assume pos is a valid access path for an insert_T operation on T . Let $T[pos[< i]] = (v, L)$.

$$\begin{aligned} & \text{insert}_T(\text{insert}_T(t, pos[> i], T[pos[\leq i]]), pos[\leq i], \text{delete}_T(pos[\leq i], T)) \\ \stackrel{IH}{=} & \text{insert}_T(\text{insert}_T(\text{insert}_T(t, pos[> i], T[pos[\leq i]]), (pos[\leq i])[\geq i], \\ & \quad \text{delete}_T(pos[\leq i], T)[pos[< i]]), \\ & \quad (pos[\leq i])[\leq i], \text{delete}_T(pos[\leq i], \text{delete}_T(pos[\leq i], T))) \\ \stackrel{D4}{=} & \text{insert}_T(\text{insert}_T(\text{insert}_T(t, pos[> i], T[pos[\leq i]]), [pos[i]], \\ & \quad \text{delete}_T(pos[\leq i], T)[pos[< i]]), \\ & \quad pos[\leq i], \text{delete}_T(pos[\leq i], T)) \\ \stackrel{DT, D4}{=} & \text{insert}_T(\text{insert}_T(\text{insert}_T(t, pos[> i], T[pos[\leq i]]), [pos[i]], \\ & \quad (v, \text{delete}_L(pos[i], L))), \\ & \quad pos[\leq i], \text{delete}_T(pos[\leq i], T)) \\ \stackrel{DT, D3}{=} & \text{insert}_T((v, \text{insert}_L(\text{insert}_T(t, pos[> i], L[pos[i]]), pos[i], \\ & \quad \text{delete}_L(pos[i], L))), \\ & \quad pos[\leq i], \text{delete}_T(pos[\leq i], T)) \\ \stackrel{D3}{=} & \text{insert}_T(\text{insert}_T(t, pos[\geq i], (v, L)), pos[< i], \text{delete}_T(pos[< i], T)) \\ \stackrel{IH}{=} & \text{insert}_T(t, pos, T) \end{aligned}$$

□

Lemma 2. *Given a tree T and a valid access path pos for a delete_T operation. For all $0 < i < |pos|$ the following statement is always true:*

$$\text{delete}_T(pos, T) = \text{insert}_T(\text{delete}_T(pos[\geq i], T[\llbracket pos[< i] \rrbracket]), pos[< i], \text{delete}_T(pos[< i], T))$$

Proof of Lemma 2. We can easily prove Lemma 2 with induction over i similar to the proof of Lemma 1. □

In Lemma 3 we state, that an insert_T operation is eliminated by a delete_T operation if both operations have the same access path and that if we access a tree at a position where we just inserted an element, we receive exactly that element.

Lemma 3. *For a given tree T and a valid access path pos for an insert_T operation we know that:*

1. $\forall t. \text{delete}_T(pos, \text{insert}_T(t, pos, T)) = T$
2. $\forall t. \text{insert}_T(t, pos, T)[\llbracket pos \rrbracket] = t$

Proof of Lemma 3. The claims follow directly from the definitions of insert_T and delete_T . □

3.3 Operational Transformation and Transformation Functions

We consider an *operation* to be an identifier together with zero or more input values and well defined semantics. For example the insert_L operation has three input values: an item to be inserted, a position and a list. The semantics is given by the implementation. Sometimes we consider instances of an operation i.e., operations with given input values. For example $O_1 = \text{insert}_L(a, 2, [x, y, z])$. In this case we call the list $[x, y, z]$ the *context* of this instance since the result would be a new context for another instance. We denote the context of one instance O_1 as $C(O_1)$. For the rest of this work we omit the term *instance* and just say operation, even if concrete values are present.

Definition 6 (Operation Composition). *Given two operations O_1 and O_2 where the context of O_2 is the result of O_1 . We write $O_2 \circ O_1$ for the composition of the operations where O_2 is applied after O_1 has been applied.*

We write $O_2 \circ O_1(X)$ to express that the context of O_1 is X .

Definition 7 (Transformation Function). *A Transformation Function has a pair of operation instances (O_1, O_2) with $C(O_1) = C(O_2)$ as input parameter and returns a pair of transformed operation instances (O'_1, O'_2) where O'_1 is the transformed version of O_1 with $C(O'_1) = O_2$ and O'_2 is the transformed version of O_2 with $C(O'_2) = O_1$.*

Our definition of the transformation function is based on the introduction of the Jupiter OT system [6]. The input of a transformation function can be seen as two independent operations (more concrete: instances) of two sites. Both sites

apply the operation to a local replica of a context which may result in inconsistent replicas over both sites. In order to achieve consistent replicas, the transformed versions of the operations are exchanged and applied by both sites.

In Listing 1 we present one transformation function for list operations XFORM_L which was initially introduced by Ellis and Gibbs in [2] and slightly improved by Ressel et al in [8]. In the listing we omit the last parameter of each operation since the context of all operations is defined in Def. 7. We implemented a proof in the interactive theorem prover Isabelle to verify that the transformation is correct and that the replicas will be consistent eventually³.

The transformation of the "XYZ" example from the introduction would be processed in the lines 7 and 13 of Listing 1. According to line 4 we need to use application dependent priorities to transform two insert_L operations with identical position parameters. For example we can define a priority where operations of one site are prioritized over all other operations. If we have two delete_L operations with identical position parameters (see line 19), both sites independently delete the same element from the list which result in a consistent state. Hence we transform both operations to no-op.

One essential property of the transformation function to achieve consistent replicas is the *Transformation Property 1* (TP1) [8]. In essence TP1 describes that the transformation function needs to repair the inconsistencies that occur if two operation instances are applied in different orders.

Definition 8 (Transformation Property 1). *Let O_1 and O_2 be two arbitrary operations with the same context $C(O_1) = C(O_2) = C$. A transformation function XFORM satisfies the Transformation Property 1 (TP1), if the following holds for $\text{XFORM}(O_1, O_2) = (O'_1, O'_2)$:*

$$O'_2 \circ O_1(C) = O'_1 \circ O_2(C)$$

Listing 1: Pseudo code of the transformation function XFORM_L

```

1 function XFORML(insertL(i1, k1), insertL(i2, k2)):
2   if k1 < k2: return(insertL(i1, k1), insertL(i2, k2 + 1))
3   if k1 > k2: return(insertL(i1, k1 + 1), insertL(i2, k2))
4   if k1 == k2: # use application dependent priorities
5
6 function XFORML(insertL(i, k1), deleteL(k2)):
7   if k1 < k2: return(insertL(i, k1), deleteL(k2 + 1))
8   if k1 > k2: return(insertL(i, k1 - 1), deleteL(k2))
9   if k1 == k2: return(insertL(i, k1), deleteL(k2 + 1))
10
11 function XFORML(deleteL(k1), insertL(i, k2)):
12   if k1 < k2: return(deleteL(k1), insertL(i, k2 - 1))
13   if k1 > k2: return(deleteL(k1 + 1), insertL(i, k2))
14   if k1 == k2: return(deleteL(k1 + 1), insertL(i, k2))
15
16 function XFORML(deleteL(k1), deleteL(k2)):
17   if k1 < k2: return(deleteL(k1), deleteL(k2 - 1))
18   if k1 > k2: return(deleteL(k1 - 1), deleteL(k2))
19   if k1 == k2: return(no-op, no-op)

```

³<https://gitlab.tubit.tu-berlin.de/jungnickel/isabelle>

4 Tree Transformations

In this section we develop a transformation function for n -ary tree operations. Therefore we introduce the definition of the transformation point and construct a transformation function that satisfies TP1. The proof that our transformation function satisfies the property is stated in the appendix. We maintain a very high level of detail, since the transformation of tree operations requires a precise definition of the transformed access paths.

Definition 9 (Transformation Point). *Given two non empty lists l_1 and l_2 of natural numbers. The Transformation Point (TPt) is the index of the first difference of l_1 and l_2 . If l_1 is a sublist of l_2 , the Transformation Point is the index of the last element of l_1 . If l_2 is a sublist of l_1 , the Transformation Point is the index of the last element of l_2 .*

If we consider two tree operations, the transformation point marks the point where a transformation may be necessary. We give two short examples of the transformation point:

$$\text{TPt}([1, 2, 3], [1, 2, 4]) = 2 \quad \text{TPt}([1, 0], [1, 0, 3, 2]) = 1$$

With the transformation point we determine whether two operations are effect dependent or effect independent i.e., if a transformation is necessary or not. We provide a definition for the effect independent tree operations in Def. 10.

Definition 10 (Effect Independence of Tree Operations). *Let pos_1 and pos_2 be the access paths of the operations O_1 and O_2 and tp be the transformation point of pos_1 and pos_2 . The operations O_1 and O_2 are effect independent tree operations, denoted by $O_1 \parallel O_2$, iff:*

1. $(|pos_1| > (tp + 1)) \wedge (|pos_2| > (tp + 1))$
2. $(pos_1[tp] > pos_2[tp]) \wedge (|pos_1| < |pos_2|)$
3. $(pos_1[tp] < pos_2[tp]) \wedge (|pos_1| > |pos_2|)$

The three cases of Def. 10 are visualized for two insert_T operations in Fig. 3. The trees t_1 and t_2 are the subtrees which should be inserted by the two insert_T operations O_1 and O_2 . The effect of the operation O_1 , that is the insertion of t_1 , is visualized as a blue circle. The effect of the operation O_2 is visualized as a red circle. Note that the Transformation Point in all examples is 0. In the left tree we demonstrate the first case of Def. 10. Both trees t_1 and t_2 are inserted in nodes which are beyond the transformation point. The trees in the middle and in the right represent the second and third case of Def. 10. In these cases one tree is inserted into a node left to the position where the other tree is inserted. We note that the order of two effect independent operations does not matter.

Definition 11 (update⁺). *The function update^+ has two input parameters: an access path pos and a number n . The result is a modified access path, where the n^{th} element of pos is increased by 1.*

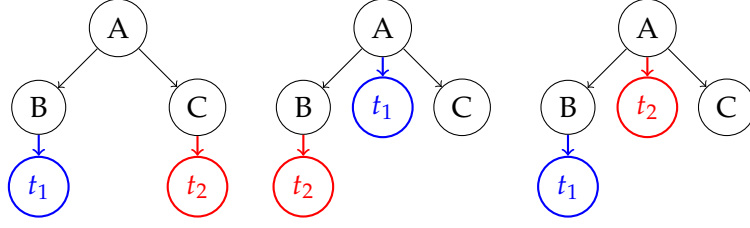


Figure 3: Demonstration of the cases of effect independent tree operations in Def. 10

Listing 2: Pseudo code of the transformation of insert_T against insert_T

```

1 function XFORMT(insertT(t1, pos1), insertT(t2, pos2)):
2   TP = TPT(pos1, pos2)
3
4   if effectIndependent(pos1, pos2):
5     return(insertT(t1, pos1), insertT(t2, pos2))
6
7   if pos1[TP] > pos2[TP]:
8     return(insertT(t1, update+(pos1, TP)), insertT(t2, pos2))
9
10  if pos1[TP] < pos2[TP]:
11    return(insertT(t1, pos1), insertT(t2, update+(pos2, TP)))
12
13  if pos1[TP] == pos2[TP]:
14    if len(pos1) > len(pos2):
15      return(insertT(t1, update+(pos1, TP)), insertT(t2, pos2))
16
17    if len(pos1) < len(pos2):
18      return(insertT(t1, pos1), insertT(t2, update+(pos2, TP)))
19
20  if pos1 == pos2:
21    # use application dependent priorities

```

Definition 12 (update^-). *The function update^- has two input parameters: an access path pos and a number n . The result is a modified access path, where the n^{th} element of pos is decreased by 1.*

The functions update^+ and update^- are used in the transformation functions to modify an access path of an operation at a specific position. We present the pseudo code for the transformation of an insert_T operation against one insert_T operation as XFORM_T in Listing 2. Transforming one insert_T operation against another one is similar to the transformation of two insert_L list operations in XFORM_L . Particularly, in XFORM_T we perform exact the same transformation at the transformation point as in XFORM_L , only the items of the lists are now subtrees. First we check whether we need a transformation i.e., if both operations are effect independent as defined in Def. 10. Then we check similar to Listing 1 how the position parameters at the transformation point are related and transform the position parameter at the transformation point as in XFORM_L . If we need to transform one insert_T operation against another one with an identical access path, application dependent priorities are used to privilege one operation.

Listing 3: Pseudo code of the transformation of delete_T against delete_T

```

1 function XFORMT(deleteT(pos1), deleteT(pos2)):
2   TP = TPt(pos1, pos2)
3
4   if effectIndependent(pos1, pos2):
5     return(deleteT(pos1), deleteT(pos2))
6
7   if pos1[TP] > pos2[TP]:
8     return(deleteT(update-(pos1, TP)), deleteT(pos2))
9
10  if pos1[TP] < pos2[TP]:
11    return(deleteT(pos1), deleteT(update-(pos2, TP)))
12
13  if pos1[TP] == pos2[TP]:
14    if len(pos1) > len(pos2): # delete from a deleted tree
15      return(no-op, deleteT(pos2))
16    if len(pos1) < len(pos2): # delete from a deleted tree
17      return(deleteT(pos1), no-op)
18    if pos1 == pos2:
19      return(no-op, no-op)

```

We introduce a transformation function for two delete_T operations in Listing 3. The main difference to the previous transformation of insert_T operations is that we decrement the position parameters at the transformation point as in XFORM_L . Therefore we use the function update^- . If the transformation point of both delete_T operations is equal we either delete a subtree from an already deleted subtree or we have two identical position parameters. Both variants are handled with no-op operations.

After introducing transformation functions for two insert_T or two delete_T operations, we combine both functions to achieve a transformation function for a transformation of insert_T against delete_T . We state the last transformation function in Listing 4. In the transformation function we modify the access paths exactly as shown in the previous XFORM_L functions. We observe one special case if the access paths of both operations at the transformation point are identical. If in this case the access path of the insert_T operation contains more items than the access path of the delete_T operation, one tree is inserted into a deleted tree (corresponding lines 14-15). To solve this conflict, we use a no-op operation as shown in Listing 3.

The transformation of a delete_T operation against an insert_T operation can be directly derived from Listing 4. The only difference to the transformation of the insert_T against delete_T is that the input parameters as well as the return parameters are interchanged.

Ultimately we introduced a transformation function for every combination of insert_T and delete_T operations that fits exactly our needs i.e., the synchronization of edits on generic hierarchical objects. Moreover the stated transformation function is correct with respect to the necessary property (TP1) to guarantee consistent results. We present our formal proof in following subsection.

Listing 4: Pseudo code of the transformation of insert_T against delete_T

```

1 function XFORMT(insertT(t, pos1), deleteT(pos2)):
2   TP = TPt(pos1, pos2)
3
4   if effectIndependent(pos1, pos2):
5     return(insertT(t, pos1), deleteT(pos2))
6
7   if pos1[TP] > pos2[TP]:
8     return(insertT(t, update-(pos1, TP)), deleteT(pos2))
9
10  if pos1[TP] < pos2[TP]:
11    return(insertT(t, pos1), deleteT(update+(pos2, TP)))
12
13  if pos1[TP] == pos2[TP]:
14    if len(pos1) > len(pos2): # insert into deleted tree
15      return(no-op, deleteT(pos2))
16    else:
17      return(insertT(t, pos1), deleteT(update+(pos2, TP)))

```

4.1 Proofs

Since our transformation function is composed of one function for each pair out of $\{\text{insert}_T, \text{delete}_T\}$, we proof the TP1 validity for each pair separately.

Lemma 4. *The transformation function XFORM_T satisfies TP1 for the transformation of an insert_T against an insert_T operation.*

Proof of Lemma 4. Let (t_1, pos_1, T) and (t_2, pos_2, T) be arbitrary but valid input parameters for two insert_T operations and let O'_1 and O'_2 be the result of the transformation of $O_1 = \text{insert}_T(t_1, \text{pos}_1, T)$ against $O_2 = \text{insert}_T(t_2, \text{pos}_2, T)$:

$$\text{XFORM}_T(O_1, O_2) = (O'_1, O'_2)$$

We show, that XFORM_T satisfies the TP1 property:

$$O'_2 \circ O_1(T) = O'_1 \circ O_2(T)$$

Let $tp = \text{TPt}(\text{pos}_1, \text{pos}_2)$ be the transformation point and let

$$s = T \llbracket \text{pos}_1[< tp] \rrbracket = (v, L)$$

be the subtree at position $\text{pos}_1[< tp]$. Note that $\text{pos}_1[\leq tp]$ and $\text{pos}_2[\leq tp]$ directing to a position in L and $\text{pos}_1[< tp] = \text{pos}_2[< tp]$. Let T' be the original tree T without the subtree s . For each if-statement in XFORM_T we consider the proof separately and highlight the corresponding lines in Listing 2 for each case.

Case 1. $O_1 \parallel O_2$ (corresponding line 2)

If O_1 and O_2 are effect independent, the order of the execution does not influence the result. According to XFORM_T both operations are left untouched during the transformation. We show that $O_2 \circ O_1(T) = O_1 \circ O_2(T)$.

We have already given some intuition for effect independent cases in Fig. 3. Next we prove that the execution order is independent for such operations.

Let $T_1 = \text{insert}_T(t_1, \text{pos}_1, T)$ be the tree, where the subtree

$$T[\llbracket \text{pos}_1[< |\text{pos}_1| - 1] \rrbracket] = (v_1, L_1)$$

is replaced by $(v_1, \text{insert}_L(t_1, \text{pos}_1[\llbracket \text{pos}_1| - 1 \rrbracket], L_1))$. Note that the subtree (v_1, L_1) is the subtree where t_1 should be inserted into.

Let $T_2 = \text{insert}_T(t_2, \text{pos}_2, T)$ be the subtree where

$$T[\llbracket \text{pos}_2[< |\text{pos}_2| - 1] \rrbracket] = (v_2, L_2)$$

is replaced by $(v_2, \text{insert}_L(t_2, \text{pos}_2[\llbracket \text{pos}_2| - 1 \rrbracket], L_2))$. We restate the proof goal for this case to:

$$\text{insert}_T(t_2, \text{pos}_2, T_1) = \text{insert}_T(t_1, \text{pos}_1, T_2)$$

Consider the tree $\text{insert}_T(t_2, \text{pos}_2, T_1)$. We expect a new tree i.e., the result, where the subtree

$$T_1[\llbracket \text{pos}_2[< |\text{pos}_2| - 1] \rrbracket] = (v'_2, L'_2)$$

is replaced by $(v'_2, \text{insert}_L(t_2, \text{pos}_2[\llbracket \text{pos}_2| - 1 \rrbracket], L'_2))$. Because of the conditions in Def. 10 we note that for any non-empty sublist pos'_2 of pos_2 we have neither inserted nor deleted a tree to a position left of $\text{pos}'_2[\llbracket \text{pos}'_2| - 1 \rrbracket]$. In contrast to situations where we have two insert_L operations on the same list, we have never performed a “shift” during the insertion of t_1 . Hence $(v'_2, L'_2) = (v_2, L_2)$. We obtain a tree where (v_1, L_1) is replaced by

$$(v_1, \text{insert}_L(t_1, \text{pos}_1[\llbracket \text{pos}_1| - 1 \rrbracket], L_1))$$

and (v_2, L_2) is replaced by

$$(v_2, \text{insert}_L(t_2, \text{pos}_2[\llbracket \text{pos}_2| - 1 \rrbracket], L_2))$$

From this point we can analogously show that the obtained tree is identical to $\text{insert}_T(t_1, \text{pos}_1, T_2)$. Thus $O_2 \circ O_1(T) = O_1 \circ O_2(T)$.

Case 2. $\text{pos}_1[tp] > \text{pos}_2[tp]$ (corresponding lines 7-8)

We have the transformed operations:

$$\begin{aligned} O'_1 &= \text{insert}_T(t_1, \text{update}^+(\text{pos}_1, tp), O_1) \\ O'_2 &= \text{insert}_T(t_2, \text{pos}_2, O_2) \end{aligned}$$

Since O_1 and O_2 are not effect dependent, we know that $\text{pos}_2[tp]$ refers to the last element of pos_2 . Hence we restate our proof goal:

$$\begin{aligned} &\text{insert}_T(t_2, \text{pos}_2, \text{insert}_T(t_1, \text{pos}_1, T)) \\ &= \text{insert}_T(t_1, \text{update}^+(\text{pos}_1, tp), \text{insert}_T(t_2, \text{pos}_2, T)) \end{aligned}$$

We divide the proof goal into two sub cases since insert_T is recursively defined in Def. 3. Both cases are visualized in Fig. 4. In this example the transformation point is 0 and from $\text{pos}_1[tp] > \text{pos}_2[tp]$ we know that t_1 is inserted at a position right of t_2 . We show the goals directly by using the presented lemmata and definitions.

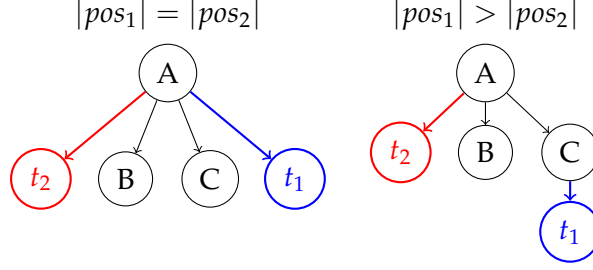


Figure 4: Example situations for case 2 in the proof of Lemma 4

Case 2.1. $|pos_1| = |pos_2|$

In this case we insert the subtrees t_1 and t_2 into the subtree s .

$$\begin{aligned}
& \text{insert}_T(t_2, pos_2, \text{insert}_T(t_1, pos_1, T)) \\
\stackrel{L_1}{=} & \text{insert}_T(t_2, pos_2, \text{insert}_T(\text{insert}_T(t_1, pos_1[\geq tp], s), pos_1[< tp], T')) \\
\stackrel{D_3}{=} & \text{insert}_T(t_2, pos_2, \text{insert}_T((v, \text{insert}_L(t_1, pos_1[tp], L)), pos_1[< tp], T')) \\
\stackrel{L_1}{=} & \text{insert}_T(\text{insert}_T(t_2, pos_2[\geq tp], (v, \text{insert}_L(t_1, pos_1[tp], L))), pos_1[< tp], T') \\
\stackrel{D_3}{=} & \text{insert}_T((v, \text{insert}_L(t_2, pos_2[tp], \text{insert}_L(t_1, pos_1[tp], L))), pos_1[< tp], T') \\
\stackrel{TP_1}{=} & \text{insert}_T((v, \text{insert}_L(t_1, pos_1[tp] + 1, \text{insert}_L(t_2, pos_2[tp], L))), pos_1[< tp], T') \\
\stackrel{D_3}{=} & \text{insert}_T(\text{insert}_T(t_1, \text{update}^+(pos_1, tp)[\geq tp], (v, \text{insert}_L(t_2, pos_2[tp], L))), \\
& \quad pos_2[< tp], T') \\
\stackrel{L_1}{=} & \text{insert}_T(t_1, \text{update}^+(pos_1, tp), \text{insert}_T((v, \text{insert}_L(t_2, pos_2[tp], L)), \\
& \quad pos_2[< tp], T')) \\
\stackrel{D_3}{=} & \text{insert}_T(t_1, \text{update}^+(pos_1, tp), \text{insert}_T(\text{insert}_T(t_2, pos_2[\geq tp], s), \\
& \quad pos_2[< tp], T')) \\
\stackrel{L_1}{=} & \text{insert}_T(t_1, \text{update}^+(pos_1, tp), \text{insert}_T(t_2, pos_2, T))
\end{aligned}$$

In the 4th line of the proof we referring to the TP1 validity of XFORM_L.

Case 2.2. $|pos_1| > |pos_2|$

In this case we insert the subtree t_2 into the subtree s and replace the tree at position $pos_1[\leq tp]$ with a new subtree which contains t_1 . Let L' be the list of subtrees L without the subtree at position $pos_1[\leq tp]$ and let s_{O_1} be the subtree at position $pos_1[\leq tp]$ after executing O_1 to the tree.

$$\begin{aligned}
L' &= \text{delete}_L(pos_1[tp], L) \\
s_{O_1} &= \text{insert}_T(t_1, pos_1, T) \llbracket pos_1[\leq tp] \rrbracket \\
&= \text{insert}_T(t_1, pos_1[> tp], L[pos_1[tp]]) \\
&= \text{insert}_T(t_1, pos_1[> tp], \text{insert}_L(t_2, pos_2[tp], L)[pos_1[tp] + 1])
\end{aligned}$$

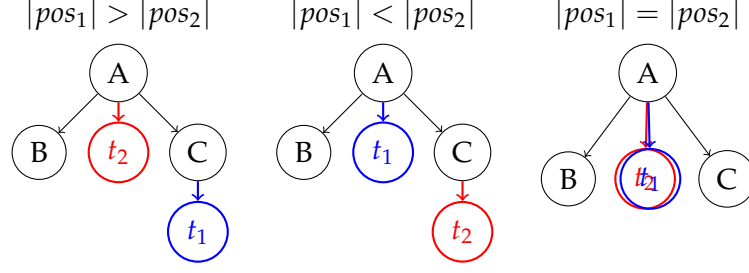


Figure 5: Example situations for case 4 in the proof of Lemma 4

We show our proof goal directly by using the presented lemmata and definitions.

$$\begin{aligned}
& \text{insert}_T(t_2, pos_2, \text{insert}_T(t_1, pos_1, T)) \\
\stackrel{L1}{=} & \text{insert}_T(t_2, pos_2, \text{insert}_T(\text{insert}_T(t_1, pos_1[\geq tp], s), pos_1[< tp], T')) \\
\stackrel{D3}{=} & \text{insert}_T(t_2, pos_2, \text{insert}_T((v, \text{insert}_L(s_{O_1}, pos_1[tp], L')), pos_1[< tp], T')) \\
\stackrel{L1}{=} & \text{insert}_T(\text{insert}_T(t_2, pos_2[\geq tp], (v, \text{insert}_L(s_{O_1}, pos_1[tp], L'))), pos_1[< tp], T') \\
\stackrel{D3}{=} & \text{insert}_T((v, \text{insert}_L(t_2, pos_2[tp], \text{insert}_L(s_{O_1}, pos_1[tp], L'))), pos_1[< tp], T') \\
\stackrel{TP1}{=} & \text{insert}_T((v, \text{insert}_L(s_{O_1}, pos_1[tp] + 1, \\
& \quad \text{insert}_L(t_2, pos_2[tp], L'))), pos_1[< tp], T') \\
\stackrel{TP1}{=} & \text{insert}_T((v, \text{insert}_L(s_{O_1}, pos_1[tp] + 1, \\
& \quad \text{delete}_L(pos_1[tp] + 1, \text{insert}_L(t_2, pos_2[tp], L'))), pos_1[< tp], T') \\
\stackrel{D3}{=} & \text{insert}_T(\text{insert}_T(t_1, \text{update}^+(pos_1, tp)[\geq tp], \\
& \quad (v, \text{insert}_L(t_2, pos_2[tp], L))), pos_2[< tp], T') \\
\stackrel{L1}{=} & \text{insert}_T(t_1, \text{update}^+(pos_1, tp), \\
& \quad \text{insert}_T((v, \text{insert}_L(t_2, pos_2[tp], L)), pos_2[< tp], T')) \\
\stackrel{D3}{=} & \text{insert}_T(t_1, \text{update}^+(pos_1, tp), \\
& \quad \text{insert}_T(\text{insert}_T(t_2, pos_2[\geq tp], s), pos_2[< tp], T')) \\
\stackrel{L1}{=} & \text{insert}_T(t_1, \text{update}^+(pos_1, tp), \text{insert}_T(t_2, pos_2, T))
\end{aligned}$$

Case 3. $pos_1[tp] < pos_2[tp]$ (corresponding lines 10-11)

This case is analog to the previous case.

Case 4. $pos_1[tp] = pos_2[tp]$ (corresponding lines 13-21)

The scenarios of this case are demonstrated in Fig. 5. We analyze the three cases separately.

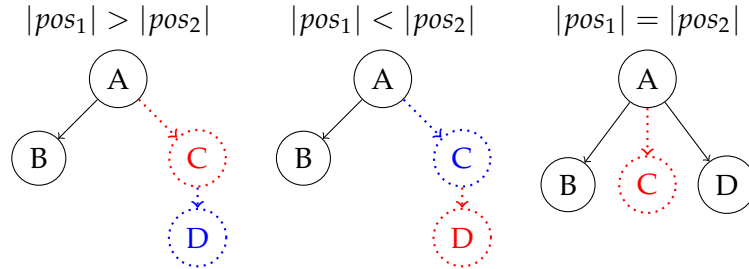


Figure 6: Examples for case 4 in the proof of Lemma 5

Case 4.1. $|pos_1| > |pos_2|$ (corresponding lines 14-15)

In this case we insert the tree t_2 directly left to the subtree s_{O_1} (the subtree at position $pos_1[\leq tp]$ after applying O_1 to T). This case is analog to case 2.

Case 4.2. $|pos_1| < |pos_2|$ (corresponding lines 17-18)

In this case we insert the tree t_1 directly left to the subtree s_{O_2} (the subtree at position $pos_2[\leq tp]$ after applying O_2 to T). This case is analog to case 3.

Case 4.3. $|pos_1| = |pos_2|$ (corresponding lines 20-21)

In this case we need to insert the trees t_1 and t_2 directly to the same position. We necessarily need to decide which operation should be preferred. We can use application dependent priorities to handle the case as in case 2 or case 3. $\square$

For the following proofs we reduce the level of detail since the substantial parts are demonstrated in the proof of Lemma 4.

Lemma 5. *The transformation function $XFORM_T$ satisfies TP1 for the transformation of a $delete_T$ against a $delete_T$ operation.*

Proof of Lemma 5. We have two delete operations O_1 and O_2 with $O_1 = delete_T(pos_1, T)$ and $O_2 = delete_T(pos_2, T)$ and the operations from the transformation function $XFORM_T(O_1, O_2) = (O'_1, O'_2)$ and divide the proof in one case per if-statement in Listing 3.

Case 1. $O_1 \parallel O_2$ (corresponding line 2)

The proof of this case is analog to the proof of the corresponding case in Lemma 4, except the $insert_L$ operations are exchanged by $delete_L$ operations.

Case 2. $pos_1[tp] > pos_2[tp]$ (corresponding lines 7-8)

After applying the transformation function we can restate the precise proof goal to:

$$\begin{aligned} & delete_T(pos_2, delete_T(pos_1, T)) \\ = & delete_T(update^-(pos_1, tp), delete_T(pos_2, T)) \end{aligned}$$

As in the previous lemma, we get two sub cases for from the definition of $delete_T$. The two sub cases are illustrated in Fig. 7. Either we delete two trees from the same list or we delete a tree from a tree right to the deleted subtree. The effect of O_1 is visualized by a dotted blue circle and O_2 is visualized by a dotted red circle. We use Lemma 2, Def. 4 and the Transformation Property 1 of $XFORM_L$ to show the proof goal directly for each case.

Case 3. $pos_1[tp] < pos_2[tp]$ (corresponding lines 10-11)

This case is analog to the previous case. Only the operations are interchanged.

Case 4. $pos_1[tp] = pos_2[tp]$ (corresponding lines 13-19)

There are three scenarios for this case which are demonstrated in Fig. 6. In the first scenario the operation O_1 deletes a tree from a tree that is deleted by O_2 . According to the transformation function the O_1 is eliminated and will be transformed to no-op. The second scenario is identical only the operations are interchanged. In the third scenario we have identical position parameters for O_1 and O_2 . Hence one

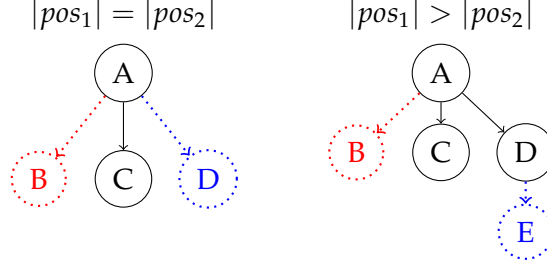


Figure 7: Examples for case 2 in the proof of Lemma 5

tree is deleted by both operations. According to the transformation function we achieve a consistent result if both operations are transformed to no-op. The proof of this case is analog to the proof of the corresponding case in Lemma 4. $\square$

Since the transformation function for the transformation of a delete_T operation against an insert_T operation and vice versa is derived from Listing 2 and Listing 3, we omit the proof of TP1 for this case since the substantial parts are already shown in the previous lemmata. Ultimately we declare the final theorem.

Theorem 1. *The transformation function XFORM_T satisfies TP1.*

Proof of Theorem 1. The transformation of insert_T against insert_T satisfies TP1 as shown in Lemma 4. From Lemma 5 we know that the transformation of delete_T against delete_T satisfies TP1 as well. The proof of TP1 in the transformation of insert_T against delete_T and delete_T against insert_T is analog to the proof of Lemma 4 and 5. Hence all cases of XFORM_T are satisfying TP1. $\square$

5 Conclusion

With the introduced transformation functions for ordered n -ary trees we presented a fruitful way to use operational transformation to synchronize replicas of such structures in an optimistic and comfortable way. Since we have focused on such generic data types, the range of applications that could implement our ideas is wide spread. The transformation function for trees can easily be adopted to other hierarchal architectures like XML documents, which are commonly used in web services and other applications to store and exchange objects.

We have analyzed the correctness of the transformation functions for lists and n -ary trees. All analyzed transformation functions satisfy TP1 which is essential for a successful synchronization with operational transformation. In addition we implemented a proof of TP1 for the transformation function for lists in the interactive theorem prover Isabelle/HOL so we can ensure that our results are correct up to the correctness of Isabelle/HOL.

As future work, the proof of TP1 for the transformation function for trees can be implemented in Isabelle/HOL to ensure the correctness of our proof up to the correctness of Isabelle/HOL. In addition our transformation functions can be

integrated in an OT programming framework so that applications can benefit from our approach.

One more complex future work is the enhancement of the transformation functions so that the more complex Transformation Property 2 (TP2) is satisfied. If TP2 is satisfied, more algorithms and less restrictive algorithms can be used to synchronize replicas of trees.

References

- [1] A. H. Davis, C. Sun, and J. Lu. Generalizing operational transformation to the standard general markup language. In *Computer Supported Cooperative Work, CSCW '02*, pages 58–67, 2002.
- [2] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. *SIGMOD Rec.*, 18(2):399–407, 1989.
- [3] A. Imine, P. Molli, G. Oster, and M. Rusinowitch. Proving correctness of transformation functions in real-time groupware. In *European Conference on Computer Supported Cooperative Work, ECSCW '03*, pages 277–293, Norwell, MA, USA, 2003.
- [4] A. Imine, M. Rusinowitch, G. Oster, and P. Molli. Formal design and verification of operational transformation algorithms for copies convergence. *Theoretical Computer Science*, 351(2):167 – 183, 2006.
- [5] T. Jungnickel and T. Herb. Simultaneous editing of json objects via operational transformation. In *ACM Symposium of Applied Computing, SAC '16*, 2016.
- [6] D. A. Nichols, P. Curtis, M. Dixon, and J. Lamping. High-latency, low-bandwidth windowing in the jupiter collaboration system. In *Symposium on User Interface and Software Technology, UIST '95*, pages 111–120, 1995.
- [7] G. Oster, H. Skaf-Molli, P. Molli, and H. Naja-Jazzar. Supporting Collaborative Writing of XML Documents. In *Enterprise Information Systems: Software Agents and Internet Computing*, pages 335–342, 2007.
- [8] M. Ressel, D. Nitsche-Ruhland, and R. Gunzenhäuser. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *Computer Supported Cooperative Work, CSCW '96*, pages 288–297, 1996.
- [9] H. Skaf-Molli, P. Molli, C. Rahhal, and H. Naja-Jazzar. Collaborative writing of xml documents. In *Information and Communication Technologies: From Theory to Applications, ICTTA '08*, pages 1–6, 2008.
- [10] C. Sun, S. Xia, D. Sun, D. Chen, H. Shen, and W. Cai. Transparent adaptation of single-user applications for multi-user real-time collaboration. *ACM Trans. Comput.-Hum. Interact.*, 13(4):531–582, 2006.

- [11] C. Sun, Y. Xu, and A. Agustina. Exhaustive search of puzzles in operational transformation. In *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work & Social Computing, CSCW '14*, pages 519–529, New York, NY, USA, 2014. ACM.